NTNU

Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: July 2026
Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1
Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21]. This paper proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- FHE 'holy grail' of privacy-preserving communication
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]

Structure of the thesis

- Building blocks and related works; FHE, gadget decomposition, TFHE, etc.
- Explanation of the code
- Experimental setup i.e. how the numbers where found
- Results, benchmarks, noise analysis
- Future work

Chapter 2
Related Works

This section details the fundamental building blocks of the sPAR protocol, and the protocol itself. To achieve a reasonable depth the protocol utilizes the *external product*, a primitive from TFHE bootstrapping that allows homomorphic multiplication of binary digits with near-constant noise bounds — enabling a large number of binary multiplication without consuming a level/bootstrapping. This primitive was ported to BGV-RNS for this thesis, which does not natively allow digit decomposition as normally used by the external product.

2.1 Fully Homomorphic Encryption

Homomorphic Encryption (HE) allows computation directly on encrypted data: an untrusted party can evaluate a function over ciphertexts without learning anything about the underlying plaintexts.

Modern FHE schemes operate on polynomials in the ring $\mathbb{Z}_q[X]/(\phi(m))$ where $\phi(m)$ is a cyclotomic (I denote $R_q = \mathbb{Z}_q[X]/(X^N + 1)$) and base their security on the hardness of the (decision) Ring Learning With Errors (RLWE) problem: distinguishing samples of the form $(a, a \cdot s + \epsilon)$ from uniformly random pairs in R_q^2. This section focuses on the BGV scheme [BGV11] used by the implementation of this thesis; the required TFHE primitives are presented in a later section.

Definition 2.1. Centered Residue Representation Mathematically the two groups are isomorphic, we are simply renaming $q/2 + 1 \mapsto -q/2$ etc., but in practice this affects noise bounds by keeping $\|\epsilon\| \leq q/2$ instead of $\|\epsilon\| \leq q$.

$$[a]_q = (a + q/2 \bmod q) - q/2 \longmapsto [-q/2, q/2) \tag{2.1}$$

$$\mathbb{Z}_q = \{-q/2, -q/2 + 1, \ldots, -1, 0, 1, \ldots, q/2 - 1\} \tag{2.2}$$

Definition 2.2. RLWE ciphertext $\overrightarrow{\mathbf{x}} = (b, a) \in R_q^2$ containing a uniformly random *public element* a and the masked element $b = a \cdot s + \Delta m + \epsilon$ derived from a persistent secret key s and randomly sampled noise ϵ, where Δ is the message scaling factor and $m \in R_t$ is the plaintext message being encrypted.

$$\mathsf{RLWE}(m) = (b, a) = (a \cdot s + \Delta m + \epsilon, a) \tag{2.3}$$

$$s \xleftarrow{\$} \{-1, 0, 1\}^N, \quad \epsilon \xleftarrow{\$} \chi, \quad a \xleftarrow{\$} R_q \tag{2.4}$$

An RLWE ciphertext $\overrightarrow{\mathbf{x}}$ encrypting a message m under the secret key s is denoted $\mathsf{RLWE}_s(m)$, but if the secret key is understood from context then $\mathsf{RLWE}(m)$ is typically used.

Definition 2.3. RLWE Public Key A public key is an encryption of 0 $p = (a \cdot s + \epsilon, a)$ so that encryption using the public key is as simple as adding the encoded message into

Definition 2.4. Encryption

$$\textsc{Encrypt}(m) \mapsto (a \cdot s + \textsc{Encode}(m) + \epsilon, a) \tag{2.5}$$

$$\textsc{Encrypt}(m, pk) = pk + (\textsc{Encode}(m), 0) \tag{2.6}$$

Definition 2.5. Decryption

$$\textsc{Decrypt}(x, s) = m \tag{2.7}$$

FHE schemes support homomorphic addition and multiplication, and typically admits one of the two for *free* in the sense that applying the operation directly to the ciphertexts applies it to the underlying plaintexts, without any additional machinery. Schemes based on RLWE have free addition, as seen below:

$$\vec{\mathbf{x}} + \vec{\mathbf{y}} = (b_x, a_x) + (b_y, a_y)$$
$$(a_x s + \Delta m_x + \epsilon_x, a_x) + (a_y s + \Delta m_y + \epsilon_y, a_y)$$
$$((a_x + a_y)s + \Delta(m_x + m_y) + (\epsilon_x + \epsilon_y), a_x + a_y)$$
$$\text{Let } a = (a_x + a_y) \text{ and } \epsilon = (\epsilon_x + \epsilon_y)$$
$$\therefore \vec{\mathbf{x}} + \vec{\mathbf{y}} = (as + \Delta(m_x + m_y) + \epsilon, a) = \mathsf{RLWE}_s(m_x + m_y)$$

Definition 2.6. RLWE addition

$$+ \ := \ \mathsf{RLWE}_s(m_x) + \mathsf{RLWE}_s(m_y) = \mathsf{RLWE}_s(m_x + m_y) \tag{2.8}$$

- The resulting RLWE ciphertext encodes the sum of the original messages, but also has higher noise.
- Managing noise is a central issue in FHE, especially when it comes to multiplication in Learning With Errors (LWE)-based schemes
- Homomorphic multiplication is considerably more complicated and expensive
- We mix BGV with TFHE-style external products to achieve multiplication (of binary digits), and avoid any HE multiplication.

Definition 2.7. Modulus Switching The process of converting a ciphertext $\vec{\mathbf{x}} \in R_Q^2$ encrypted under a ciphertext modulus Q into a ciphertext $\vec{\mathbf{x}}' \in R_{Q'}^2$ encrypted under a different modulus Q', while ensuring that both ciphertexts still decrypt to the same message.

Scaling the message by a factor of Q'/Q and rounding to the nearest integer achieves this,

This process is typically the primary tool for noise management in leveled schemes, as scaling down the modulus also scales down the noise, allowing further computation without bootstrapping.

Definition 2.8. Bootstrapping The process of homomorphically evaluating the decryption circuit to reset the noise of a ciphertext, allowing further computation [Gen09]. Generally slow.

Definition 2.9. Keyswitching The process of converting a message to being encrypted under a different secret key.

Definition 2.10. Multi-Party FHE Different methods exist and are being researched. Method used in this thesis: Each party has a secret key and public key.

The full public key is each of the public key shares appended together. Noise is flooded to maintain security. Partial decryption = each client decrypts the ciphertext using their own secret key Final decryption = the server takes each partial decryption and since the decryption is linear can combine them to decrypt the message without any party knowing the (full) secret key.

2.1.1 The BGV scheme

- Introduce scheme [BGV11]
- Justify choice implicitly; strengths
  - SIMD

While most of the theory covered throughout this thesis is scheme-agnostic, the implementation is built on top of the BGV scheme [BGV11], chosen because ... which places the message in the *least* significant bits of the ciphertext by encoding the message v as a plaintext m via

$$m = \frac{\tilde{W} \cdot I_n^R \cdot v}{n} \tag{2.9}$$

Notably, $v = 0 \implies m = 0$ and $v = 1 \implies m = 1$, which is important for later.

The message scale is fixed to $\Delta = 1$ **(But some noise/scaling techniques i.e. FLEXIBLEAUTO can change Δ in practice)** and the noise is scaled by the plaintext modulus, $\epsilon = t \cdot \epsilon_{\mathsf{rlwe}}$, giving $b = a \cdot s + m + t\epsilon_{\mathsf{rlwe}}$ in (2.3). In this thesis, each RLWE ciphertext will be expressed as (2.3) to be as general as possible, even though the BGV stuff is implied.

Definition 2.11. Decryption Assuming $\epsilon < Q$, then decryption is correct.

$$\textsc{Decrypt}(x, s) = [[b - a \cdot s]_q]_t \tag{2.10}$$

Definition 2.12. SIMD Batching If the chosen plaintext modulus t is a prime satisfying the algebraic condition $t \equiv 1 \pmod{2N}$, then BGV supports per-slot SIMD packing [SV11]. This mode converts a plaintext from a polynomial of order $N - 1$ into N independent constants $[c_i]_t$. Homomorphic operations are applied to each slot individually, for example $(1, 2) \cdot (2, 3) = (2, 6)$ instead of $(1, 2) \cdot (2, 3) = (1 + 2x)(2 + 3x) = (2, 7, 6)$.

2.2 Residue Number Systems

In BGV, BFV and CKKS the ciphertext modulus Q is large, much larger than a computer word/register (typically 64 bits on modern computers). Numbers that do not fit inside a register must be handled with multi-precision arithmetic, which is known to be slow due to its serial nature that does not allow parallelization. High-performance implementations avoid this problem by using a Residue Number System (RNS) where large numbers are represented as a set of smaller, independent residues via the Chinese Remainder Theorem (CRT). RNS variants of BGV, BFV and CKKS are the standard in practice; OpenFHE, for example, only supports the RNS versions of these schemes [BAB+22].

Definition 2.13. Canonical Form We are interested in polynomials in the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ with degree (up to) $N - 1$ and coefficients in $[-Q/2, Q/2)$, and different representations of this polynomial. The canonical form (2.11) of such a polynomial a is the form most is familiar with.

$$a = c_0 + c_1 X + \cdots + c_{N-1} X^{N-1} = \sum_{i=0}^{N-1} c_i X^i \tag{2.11}$$

Definition 2.14. Coefficient Form As polynomials have a well-known/predictable structure, we can omit the redundant X variable and represent $a \in R_Q$ uniquely by its coefficients. In this thesis I will consider these forms functionally equivalent, and refer to both (2.11) and (2.12) as *canonical*.

$$a = (c_0, c_1, \ldots, c_{N-1}) \tag{2.12}$$

Let $Q = q_0 \ldots q_{k-1}$ with $\gcd(q_i, q_j) = 1 \; \forall \; i \neq j$, so that CRT allows decomposition into smaller residues. If $\max_i\{q_i\}$ fits in a computer word then we can use CRT on the coefficients of a polynomial to work exclusively with integers that fit inside a computer word.

Definition 2.15. First RNS Form (CRT Form) Given a polynomial $a \in R_Q$ where $Q = q_0 q_1 \ldots q_{k-1}$ is chosen as a product of k small primes (small meaning they each fit inside a computer register), each coefficient c_i is uniquely represented by its k residues via the Chinese Remainder Theorem. Then the **CRT form** of a (2.13) is a collection of smaller polynomials called *towers*, *residues* or *limbs*.

$$\begin{pmatrix} a \mod q_0 \\ a \mod q_1 \\ \vdots \\ a \mod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod q_0 & c_1 \bmod q_0 & \ldots & c_{N-1} \bmod q_0 \\ c_0 \bmod q_1 & c_1 \bmod q_1 & \ldots & c_{N-1} \bmod q_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod q_{k-1} & c_1 \bmod q_{k-1} & \ldots & c_{N-1} \bmod q_{k-1} \end{pmatrix} \tag{2.13}$$

Addition and subtraction of two polynomials in R_Q is equivalent to element-wise addition/subtraction of their CRT forms (modulo q_i). As each element fits within a computer word, no carries propagate between them, and all pairs of elements can be added or subtracted in parallel.

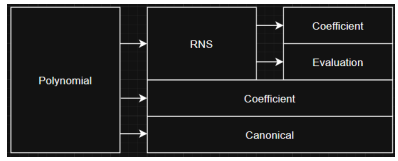
2.2.1 Number Theoretic Transform

The Number Theoretic Transform (NTT) converts a negacyclic convolution in the coefficient domain into point-wise multiplication in the NTT domain, and multiplication in R_{q_i} is exactly such a convolution of the coefficient vectors. Applying the NTT to each residue of the CRT form (2.13) therefore makes polynomial multiplication element-wise as well. The negacyclic NTT of size N requires each modulus to satisfy $q_i \equiv 1 \pmod{2N}$, which constrains the choice of RNS primes.

Definition 2.16. Second RNS form (Double CRT/NTT/Evaluation) By applying the NTT to the CRT form (2.13) we gain the ability to perform polynomial multiplication in parallel, without losing the ability to perform addition/subtraction in parallel.

$$\begin{pmatrix} \mathsf{NTT}(a \bmod q_0) \\ \mathsf{NTT}(a \bmod q_1) \\ \vdots \\ \mathsf{NTT}(a \bmod q_{k-1}) \end{pmatrix} = \begin{pmatrix} \hat{a}_{0,0} & \hat{a}_{0,1} & \dots & \hat{a}_{0,N-1} \\ \hat{a}_{1,0} & \hat{a}_{1,1} & \dots & \hat{a}_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{a}_{k-1,0} & \hat{a}_{k-1,1} & \dots & \hat{a}_{k-1,N-1} \end{pmatrix} \tag{2.14}$$

Remark 2.17. Rather confusingly, OpenFHE calls this form *Coefficient* format. Both RNS forms are represented by an `DCRTPoly` object, so a polynomial in CRT form is represented by a `DCRTPoly` object in `Format::Coefficient` mode.



The evaluation form (2.14) is the preferred form, as it supports parallel addition, subtraction and multiplication, but it still does not natively support *all* necessary operations. Because switching between canonical/vector, CRT and NTT mode is relatively expensive — requiring (inverse) CRTs and NTTs — much of the relevant literature [GHS12; BEHZ16; HPS18; KPZ21] improves upon existing functions by finding new approaches and approximations that limit the number of conversions required.

2.2.2 Base Conversion

Definition 2.18. RNS Basis. Given a large modulus $Q = \prod_{i=0}^{k-1} q_i$ composed of pairwise coprime factors, we define the *RNS basis* $\mathcal{B}_Q = \{q_0, q_1, \ldots, q_{k-1}\}$ as the set of RNS moduli used in the RNS/CRT representation (2.13).

Definition 2.19. Base conversion Let $\mathcal{B}_Q = \{q_0, \ldots, q_{k-1}\}$ and $\mathcal{B} = \{b_0, \ldots, b_{\ell-1}\}$ be RNS bases with $\gcd(Q, B) = 1$, where $B = \prod_j b_j$. Base conversion is the map that takes the representation of x in $\mathcal{B}_Q$ to the representation of $[x]_Q$ in $\mathcal{B}$, i.e.

$$\left([x]_{q_0}, \ldots, [x]_{q_{k-1}}\right) \longmapsto \left([x]_{b_0}, \ldots, [x]_{b_{\ell-1}}\right)$$

When the residues in $\mathcal{B}$ are appended to those in $\mathcal{B}_Q$, so that x becomes represented in the *extended* basis $\mathcal{B}_Q \cup \mathcal{B}$ for the modulus QB, the operation is also referred to as **base extension**.

Remark 2.20. Extend mod notation for RNS polynomials: $[a]_{\mathcal{B}_Q} = ([a]_{q_0}, \ldots, [a]_{q_{k-1}})$ for the basis $\mathcal{B}_Q = \{q_i\}_{i=0}^{k-1}$ means a must be in RNS form

The naive approach to base conversion reconstructs a from its residues through inverse CRT, and then reduces the result modulo each modulus of the new basis. This reintroduces the multi-precision arithmetic that RNS was meant to avoid. Cheaper but approximate alternatives exists, described next.

Fast Base Conversion

Introduced by [BEHZ16], fast base conversion performs operates directly on the residues (in CRT mode), avoiding the CRT reconstruction and multi-precision integers entirely. (Does not work in evaluation mode). The price is approximation, since (2.15) produces not x but $x + \alpha_x Q$ for some integer $\alpha_x \in [0, k)$. Computing the exact overflow α_x is possible but would require costly operations in RNS, and is unnecessary in *extension* by P. Introduce the notation $\hat{q}_i = Q/q_i$ here?

$$\mathsf{FastBConv}(x, \mathcal{B}_Q, \mathcal{B}) = \left(\left[\sum_{i=0}^{k-1} \left[x_i \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \times \frac{Q}{q_i} \right]_b \right)_{b \in \mathcal{B}} \tag{2.15}$$

Fast Base Extension

We can use (2.15) as a subroutine in base extension. Only the new residues $\mathcal{B}_Q \longmapsto \mathcal{B}_P$ need to be calculated, and since $\mathcal{B}_{QP} = \mathcal{B}_Q \cup \mathcal{B}_P$ the original residues are simply kept.

Remark 2.21. The factors $[(\frac{Q}{q_i})^{-1}]_{q_i}$ and $\hat{q}_i = \frac{Q}{q_i}$ can be pre-calculated and stored in memory (discussed in section 3.1).

Algorithm 2.1 Fast Base Extension

Input: $[a]^{\mathsf{NTT}}_{\mathcal{B}_Q}$ and a basis $\mathcal{B}_P$
Output: $[a+\epsilon]^{\mathsf{NTT}}_{\mathcal{B}_{QP}}$ where $\epsilon = \alpha_a Q$ for some $\alpha_a \in [0, k)$
1: $a_Q := \mathsf{InverseNTT}(a)$ ▷ Convert to CRT form
2: $a_P := \mathsf{FastBConv}(a_Q, \mathcal{B}_Q, \mathcal{B}_P)$
3: **return** $a \cup \mathsf{NTT}(a_P)$

Scale

Note that the error $\alpha_x Q \pmod{QP}$ produced by Algorithm 2.1 would be completely removed if we multiplied by P. This observation leads to the specialized operation of "extend and scale by P". I refer to this operation as $\textsc{Scale}(x, P) \mapsto [xP]_{\mathcal{B}_{QP}}$.

$$x' = x + \alpha_x Q \pmod{QP}$$
$$x'P = xP + \alpha_x QP = xP \pmod{QP}$$

$$\textsc{Scale}(x, P) = \textsc{FastBaseExtend}(x, \mathcal{B}_Q, \mathcal{B}_P) \cdot P \tag{2.16}$$

Modulus Switching

In RNS, modulus switching — also referred to as *mod down* — from Q to $Q' = Q/q_{k-1}$ amounts to *dropping the last limb* with a small correction, computed limb-wise without CRT reconstruction:

$$[c']_{q_i} = \left[q_{k-1}^{-1}\,(c - \delta)\right]_{q_i}, \quad 0 \le i < k-1, \tag{2.17}$$

where $\delta \equiv c \pmod{q_{k-1}}$ and, for BGV, $\delta \equiv 0 \pmod{t}$ so that decryption correctness is maintained. The noise shrinks by a factor q_{k-1}, matching the classical modulus switch.

Approximate Mod Down

Modulus switching exactly requires multiprecision integers.. can use FBE again... exploits the fact that the polynomial is pre-multiplied by P...

Uses $\delta = \textsc{FastBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q)$

The reverse operation, switching from the extended modulus QP back down to Q, uses (2.15) to convert the P-part back to $\mathcal{B}_Q$ before dividing by P [GHS12; HPS18]:

$$\textsc{ApproxModDown}(x) = \left[P^{-1}\,(x - \textsc{FastBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q))\right]_{\mathcal{B}_Q} \tag{2.18}$$

For BGV the converted term must additionally be corrected modulo t to preserve the plaintext [KPZ21].

2.3 Gadget Decomposition in BGV-RNS

- A problem that arises in several areas of FHE is that of multiplying a ciphertext by some large number while keeping the error low.
- The techniques covered in this section typically occur in the context of keyswitching, but we remove them entirely from that context and consider only the techniques themselves.

$$(a \cdot s + \Delta m + \epsilon) \cdot X = aX \cdot s + \Delta X m + X\epsilon \tag{2.19}$$

2.3.1 Brakerski-Vaikuntanathan

Brakerski and Vaikuntanathan [BV11] first proposed decomposing ciphertext elements into small digits as a means of controlling the noise growth of this operation. The idea has since been generalized into the idea of *gadget decomposition* and applied in a variety of forms, including digit and CRT (RNS) decomposition. Following the convention of [KPZ21; BAB+22], we refer to this family of decomposition-based techniques collectively as BV.

Definition 2.22. Digit Decomposition Given an integer $\gamma \in \mathbb{Z}_q$ for a modulus $q \geq 2$, base β and decomposition parameter ℓ, the digit decomposition $\mathsf{Decomp}(\gamma) = (\gamma_0, \gamma_1, \ldots, \gamma_{\ell-1}) \in \mathbb{Z}_\beta^\ell$ outputs an ordered list of ℓ base-β digits satisfying

$$\gamma = \gamma_0 \frac{q}{\beta} + \gamma_1 \frac{q}{\beta^2} + ...\gamma_{\ell-1} \frac{q}{\beta^\ell} = \sum_{i=0}^{\ell-1} \gamma_i \frac{q}{\beta^{i+1}}$$

Digit decomposition has two properties we care about. First, we can multiply two integers through the decomposition. By construction, $\langle \mathsf{Decomp}(\gamma), g \rangle = \gamma$ for the fixed vector $g = (q/\beta, \ldots, q/\beta^\ell)$, meaning $\langle \mathsf{Decomp}(\gamma), g \cdot x \rangle = \gamma \cdot x$ for an arbitrary integer $x \in \mathbb{Z}$. Second, if the digits γ_i of $\mathsf{Decomp}(\gamma)$ are small, then $\langle \mathsf{Decomp}(\gamma), \epsilon \rangle \leq \ell \cdot \max\{\gamma_i \cdot \epsilon_i\} \leq \beta\epsilon \ll \gamma\epsilon$. This is the mechanism that allows digit decomposition to decrease noise growth. For either of these properties, the particular form of g is unimportant as long as the digits γ_i remain small. Furthermore, as we are generally working with some intrinsic error in FHE, [CGGI18] proposed allowing some error ϵ in the reconstruction, giving the definition from their paper verbatim Theorem 2.23.

Definition 2.23. (Abstract) Gadget Decomposition An efficient algorithm $\mathsf{Decomp}^{g,\beta,\epsilon}(v)$ is a valid decomposition on the gadget $g \in \mathbb{Z}_q^\ell$ with quality $\beta \in \mathbb{Z}^+$ and precision $\epsilon \in \mathbb{Z}^+$ if and only if for any RLWE sample $v \in R_Q$ it efficiently and publicly outputs a small vector $u \in \mathbb{Z}_q$ such that $\|u\|_\infty \leq \beta$ and $\langle u, g \rangle \leq v + \epsilon$.

- Other (equivalent) form more convenient to work with, where the output of $\mathsf{Decomp}^{g,\beta,\epsilon}(v) \to D(v)$ and $g \cdot u \to P(u)$ are considered.

Definition 2.24. Gadget Property. Two vectors $P(x) \in \mathbb{Z}_Q^\ell$ and $D(y) \in \mathbb{Z}_\beta^\ell$ are said to have the *gadget property* if and only if they satisfy (2.20) for any pair of RLWE samples $(x, y) \in R_Q^2$ with precision $\epsilon \in \mathbb{Z}_q^+$ and quality β where $\|D(y)\|_\infty \leq \beta$. If $\epsilon = 0$ then $P(x)$ and $D(y)$ are said to have the *exact* gadget property.

$$\langle P(x), D(y) \rangle = x \cdot y + \epsilon \pmod{Q} \tag{2.20}$$

Lemma 2.25. Layered Gadget Decomposition *The composition of two gadget decompositions is itself a gadget decomposition.*

$$\langle P_1(u), D_1(v) \rangle = u \cdot v + \epsilon_1, \quad \langle P_2(u), D_2(v) \rangle = u \cdot v + \epsilon_2 \pmod{Q}$$
$$P_1 \circ P_2 = P_1(P_2(u)) \implies \langle P_1 \circ P_2, D_1 \circ D_2 \rangle = u \cdot v + 2\epsilon \text{ for some } D_1 \circ D_2$$

Proof. Very simply

$$\langle P_1(P_2(u)), D_1(D_2(u)) \rangle = P_2(u) \cdot D_2(u) + \epsilon_1 = u \cdot v + \epsilon_2 + \epsilon_1 \pmod{Q}$$

□

RNS Instantiation

Digit decomposition cannot be applied directly the RLWE sample in an RNS scheme, as the system only has access to the residues. When Bajard et. al. first implemented the BV cryptosystem using RNS [BEHZ16] they exchanged digit decomposition for CRT decomposition, exploiting the fact that this decomposition is already required by the RNS.

Definition 2.26. Chinese Remainder Theorem is clearly a gadget

$$CRT : \sum N \tag{2.21}$$

- CRT idempotent used in [BEHZ16]
- HPS optimization for specifically keyswitching [HPS18]
- Definition: gadget with $\beta = Q/2, \epsilon = 0$

$$P_{\mathsf{HPS}}(u) = () \tag{2.22}$$
$$D_{\mathsf{HPS}}(v) = () \tag{2.23}$$

- Note on HPS gadget = RNS limb lookup → section 3.1
- HPS is exact but noise is large ($\beta = Q/2,\ \epsilon = 0$); can apply digit decomposition to each residue polynomial as per Theorem 2.25, lowering $\beta \to B/2$ and still exact ($\epsilon = 0$) if $B = \lfloor \max q_i/\ell \rfloor + 1$

2.3.2 Gentry-Halevi-Shoup

- Another approach was implemented by Gentry, Halevi and Shoup in [GHS12].
- Generate the key in QP as $(a \cdot s + \Delta + \epsilon, a) \in \mathcal{B}_{QP}^2$
- Base extend $u \mapsto \mathcal{B}_{QP}$ and calculate inner product $\mathsf{RLWE}(mP)$
- Divide the ciphertext by P and approx mod down to $\mathcal{B}_Q$
- **FastBaseExtension and Multiply by P = add empty towers!!!**
- Since key was pre-scaled, ϵ shrinks relative to the message
- Reference RNS section, base conversion

Remark 2.27. **GHS Security** The public key is masked by elements in QP, meaning the ciphertext modulus $QP \approx 2Q$ must be used when evaluating the security. A GHS scheme has the security as a BV scheme if Q is halved.

Lemma 2.28. *GHS is a single-digit decomposition* ($\ell = 1$) *with quality* $\beta = Q/P$ *and precision* $\epsilon = 0$.

$$\langle [x]_{\mathcal{B}_{QP}}, P \cdot [y]_{\mathcal{B}_{QP}} \rangle = x \cdot y \cdot P \pmod{QP} \tag{2.24}$$

$$\langle P^{-1} \cdot [x]_{\mathcal{B}_{QP}}, P \cdot [y]_{\mathcal{B}_{QP}} \rangle = x \cdot y \cdot P \cdot P^{-1} = x \cdot y \pmod{QP} \tag{2.25}$$

$$\overset{ModDown}{\longmapsto} \quad x \cdot y \pmod{Q} \tag{2.26}$$

2.3.3 Hybrid

- BV and GHS represent a spectrum
- Hybrid = combination of the two techniques
- Group digits by d_{num}

2.4 TFHE Bootstrapping Primitives

- TFHE bootstrapping achieves fast evaluation of binary circuits.
- TFHE is optimized for computations of large multiplicative depth on small plaintexts, and is best known for its fast bootstrapping procedure.
- It is based on approximating gadget decomposition.
- Asymmetry in noise scaling of GSW ciphertexts; sum of small errors is smaller than product of large errors.
- (R)GSW ciphertexts [GSW13; DM14]
- External product [DM14; CGGI18]
  - Though digit decomposition is often used, emphasize the external product only requires $G^{-1}(x) \cdot G \approx x \bmod Q$
  - Lemma/corollary: If two vectors satisfy the gadget property (2.20), then $G = I_2 \otimes P(1)$ and $G^{-1}(x) = [D(x_0)|D(x_1)]$ is a valid choice and Algorithm 2.2 returns the external product
- Internal product [CGGI18]

Definition 2.29. RGSW Ciphertext [GSW13; DM14; CGGI18]

$$C = Z + \mu G \text{ where } G = I_2 \otimes g \tag{2.27}$$

Definition 2.30. External Product [CGGI18]

$$G^{-1}(x) \cdot G \approx x \pmod{Q} \quad \text{(gadget property)} \tag{2.28}$$

$$x \boxdot Y = G^{-1}(x) \cdot Y \tag{2.29}$$

- Gadget property vectors defines $G = I_2 \otimes P(1)$ and $G^{-1} = [D(x_0)|D(x_1)]$
- Using the decomposition $D(x)$ as a subroutine makes the external product fairly straight forward

Algorithm 2.2 External Product $\mathsf{RLWE} \boxdot \mathsf{RGSW}$ (RNS)

Input: RLWE ciphertext $(c_0, c_1) \in R_Q^2$, RGSW ciphertext $C \in R_Q^{2\ell}$
Output: RLWE ciphertext of product

```
acc ← (0, 0)
d_0 ← Decompose(c_0)
d_1 ← Decompose(c_1)
for r ← [0, ℓ) do
    acc[0] ← acc[0] + d_0[r] · C[r][0] + d_1[r] · C[r + ℓ][0]
    acc[1] ← acc[1] + d_0[r] · C[r][1] + d_1[r] · C[r + ℓ][1]
end for
return acc
```

Definition 2.31. Internal Product [CGGI18]

$$X \boxtimes Y = \begin{bmatrix} x_0 \boxdot Y \\ x_1 \boxdot Y \\ \vdots \\ x_{2\ell-1} \boxdot Y \end{bmatrix} \tag{2.30}$$

Remark 2.32. Order of operations matter; decomposition determines quality, higher noise should appear on rhs of internal product and $u = P(x)$ is always the vector with largest max norm.

2.5 (Somewhat) Practical Anonymous Router

Chapter 3
Methodology

A gap needed to be filled: implementing TFHE-style RGSW products (external and internal) in BGV-RNS. As a result of this thesis, a C++ library (`core`) was developed to bring this functionality to the OpenFHE library [BAB+22]. The sPAR protocol has also been implemented in C++ using the RGSW library. I will stress the libraries are separate; the RGSW library can be used on its own as is, and the sPAR library can in theory be used without it — provided a different library is developed to provide the same functionality and API.

- A result of this thesis: a C++ library which extends OpenFHE's BGV implementation to support RGSW operations
- And sPAR which was built on top of this library
- The code base is split into three sections found in the `lib` directory:
  1. `core` extends OpenFHE's `CryptoContext<T>` with the operations:
     - `EncryptRGSW` outputs the RGSW encryption of the input plaintext
     - `EvalExternalProduct` takes an RLWE and RGSW ciphertext
     - `EvalInternalProduct` takes two RGSW ciphertexts
     - Supporting functions like AddRGSW and MultRGSWPlaintext etc.
  2. `server` builds the sPAR server functions on top of `core`
  3. `client` builds the sPAR client functions on top of `core`
- There are also two auxiliary directories `benchmarks` and `tests` that contain code to benchmark and unit test functions, respectively.
- And `scripts` contain python code for running the estimator

This section also explains the set up of the various tests; the numbers from running these tests are presented in chapter 4.

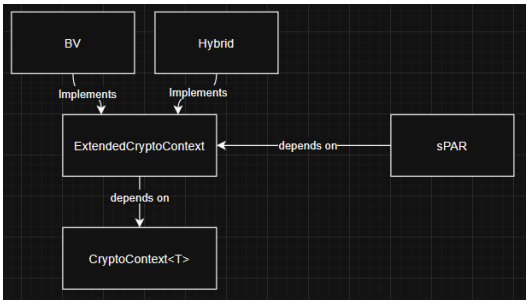
Bitwise Operations

The library assumes the base B is a power of two, $B = 2^b$, and optimizes some operations by using the exponent $b = \log B$. This allows common arithmetic performance improvements $\mathcal{O}(1)$ using bitwise operators:

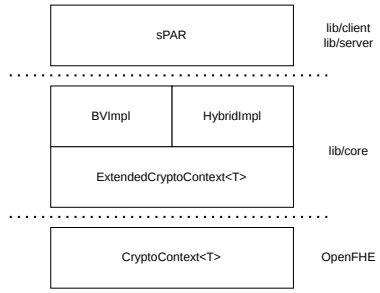
OperationNotation	Value
LeftShift(x, b): $(x \ll b)$	$x \cdot 2^b$
RightShift(x, b):$(x \gg b)$	$x/(2^b)$
division by B :$\textsc{LShift}(x, b)$	x/B
modulo: $x \bmod 2^b$	$x \wedge ((1 \ll b) - 1)$
center: $x \mapsto [x]_{2^b}$	$((x + (1 \ll (b-1)))m) - (B >> 1);$

3.1 Implementation Details of the Core Library

The core library, located in the `lib/core` directory of [**gitrepo**] provides an abstract interface named `ExtendendedCryptoContext` which provides a number of RGSW functions, including RGSW encryption, external product and internal product. Any project that requires this functionality can import the library and consume this interface without worrying about the implementation of the interface. The library is built on top of OpenFHE [BAB+22] and the interface reflects that, mirroring the API of OpenFHE. A user who is familiar with OpenFHE should as a result have no problem using this library in their own projects.



(a) Dependency graph (preview; recreate in tikz)



(b) Structure (preview)

Table 3.1: Primary functionality provided by `ExtendedCryptoContext`

$\textsc{EncryptRGSW}$(pk, pt)	Outputs an RGSW ciphertext
$\textsc{EvalExternalProduct}$(rlwe, rgsw)	Outputs the external product
$\textsc{EvalInternalProduct}$(lhs, rhs)	Outputs the internal product

The `ExtendedCryptoContext` interface comes bundled with two concrete implementations. One is designed around gadget decomposition of the RNS primes as described in section 2.3, and will be referred to as the BV implementation. The other implementation is designed around the hybrid keyswitching mechanism from subsection 2.3.3. Of these, the former should be considered the "canonical" implementation because (1) this design is the most closely related to the external product as discussed in the literature, where this product is traditionally defined using digit decomposition [**examples-of-this**; DP25], and (2) the other implementation does not use true (Ring) Gentry-Sahai-Waters (RGSW) ciphertexts.

Remark 3.1. The implementations are hard coupled to the BGV-RNS (`CryptoContextBGVRNS`) scheme. As OpenFHE provides a unified interface for both BGV and BFV — and the TFHE products are defined independent of scheme — it would not require much work to add support for BFV to the core library.

3.1.1 Brakerski-Vaikuntanathan Implementation

- Digit decomposition of each residual limb
- Two functions that output $P(v)$ and $D(u)$ with quality β s.t. $\|D(u)\|_\infty \leq B/2 = \lceil \max q_i/(2\ell) \rceil$
- The outputs has $k \times \ell$ entries, where k is the number of RNS moduli (corresponding to the first decomposition) and ℓ is the number of base B digits
- Takes only ℓ as an input parameter and computes $\log B = \lfloor \max\{\log(q_i)\}/\ell \rfloor + 1$

RGSW Encryption

- `ExtendedContextBV` implements the double decomposition; digit decomposing each RNS limb
- Let $P_g(v) = g \otimes v$ and $P_{\mathsf{BV}}(v) = P_g \circ P_{\mathsf{HPS}}(v) = g \otimes (P_{\mathsf{HPS}}(v))$

$$C = Z + \mu G = Z + \mu \cdot (I_2 \otimes P_g(P_{\mathsf{HPS}}(1)) = Z + (I_2 \otimes P_g(P_{\mathsf{HPS}}(\mu))$$
$$P_g(P_{\mathsf{HPS}}(\mu)) = g \otimes ([\mu]_{q_0}, \ldots, [\mu]_{q_{k-1}})$$
$$([\mu g]_{q_0}, \ldots [\mu g]_{q_{k-1}}) \text{ where } \mu g = (\mu, \mu B, \ldots, \mu B^\ell)$$
$$([\mu]_{q_0}, [\mu B]_{q_0}, \ldots, [\mu B^\ell]_{q_{k-1}}) \in \mathbb{Z}_Q^{k \cdot \ell}$$

- Observe that this is the same information as $(\mu, \mu B, \ldots, \mu B^\ell)$
- Because the i-th tower of μ is $[\mu]_{q_i}$, the HPS gadget row j gives $[[\mu]_{q_i}]_{q_j} = 0 \forall i \neq j$

$$\left[\begin{array}{ccc|ccc|ccc} [\mu]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & [\mu]_{q_1} & \cdots & [\mu B^{\ell-1}]_{q_1} & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & \dots & 0 & [\mu]_{q_{k-1}} & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{array}\right]$$

- Ignore all the 0 elements, then we get the exact definition of the RNS form of μ, μB, and so on
- However, each row of the RGSW must still encode only 1 tower from each of the ℓ polynomials

$$P_{\mathsf{BV}}(\mu) \overset{\text{flatten}}{\longmapsto} \begin{bmatrix} [\mu]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} \\ [\mu]_{q_1} & \cdots & [\mu B^{\ell-1}]_{q_1} \\ \vdots & \ddots & \vdots \\ [\mu]_{q_{k-1}} & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{bmatrix} = ([\mu]_{\mathcal{B}_Q}, \dots, [\mu B^{\ell-1}]_{\mathcal{B}_Q})$$

- Multiplying μ by a factor of B^i for a single i can be done in NTT
- Applying this to $Z + P_{\mathsf{BV}}(\mu)$ allows parallelization on $k\ell$ threads
- $\mathcal{B}_Q$, ℓ and B are all known/global variables tied to the context

Algorithm 3.1 Parallel RGSW-BV Encryption

Input: Decomposition parameter ℓ, base B
Input: RNS polynomial $[\mu]^{\mathsf{NTT}}_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$
Output: $\mathsf{RGSW}(\mu)$

```
Initialize empty array C with 2kℓ slots
for r ← [0, kℓ) in parallel do
    (i, j) ← (r mod ℓ, ⌊r/ℓ⌋)
    mg ← TowerMul(μ, j, B^i)
    C_r ← Encrypt(0) + (mg, 0)
    C_{r+ℓ} ← Encrypt(0) + (0, mg)
end for
return C
```

- Pre-computed value: $\textsc{GetPower}(i, j) = B^i \bmod q_j$ where the modulo is just to keep numbers smaller in memory, negligible for performance
- Then $\textsc{TowerMul}(\mu, j, B^i)$ is really just extracting the $j-th$ limb, multiplying by the pre-computed value $B^i \bmod q_j$ calculate the product modulo q_j before the tower is inserted back into μ

External Product

- **IMPORTANT:** decomposition determines the quality β, so the order of operations does matter for noise growth!
- Decomposition used = per limb digit extraction
- Must choose to decompose into $[-B/2, B/2)$ or $[0, B)$
  - Centered has lower noise as the quality is $\beta = B/2$ but more complicated
  - Unsigned is far simpler and easily parallelizable but $\beta = B$
- Centered representation creates an order; parallelization requires *pre-processing*

$$\text{Let } offset = \frac{B}{2} \cdot \frac{B^\ell - 1}{B - 1} \text{ be a global (pre-calculated) value}$$

Algorithm 3.2 Decompose wrt. B (TODO: Signed digit decomposition)

Input: RNS polynomial $[x]_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \mathbb{Z}_{q_1} \times \cdots \times \mathbb{Z}_{q_{k-1}}$, base $B = 2^b$, and decomposition parameter ℓ
Output: ℓ RNS "digit" polynomials $\mathbf{d} = (d_0, d_1, \ldots, d_{k\ell-1}) \in \mathbb{Z}_B^{k\ell}$

1: $a' \leftarrow a$
2: **for** $j = 0$ to $\ell - 1$ **do**
3: $\quad d_j \leftarrow a' \pmod B$ ▷ Extract the lower bits
4: $\quad$ **if** $d_j \geq B/2$ **then**
5: $\quad\quad d_j \leftarrow d_j - B$ ▷ Shift to signed range
6: $\quad$ **end if**
7: $\quad a' \leftarrow \frac{a' - d_j}{B}$ ▷ Shift down and apply the carry
8: **end for**
9: **return d**

- Uses the standard external product function Algorithm 2.2
- Or a slightly parallelized version: the decompose function runs in parallel, not much gain from parallizing further, the additions are already cheap/fast-ish.

Internal Product

- Trivially implemented as repeated external products as per the definition Equation 2.30

3.1.2 GHS/Hybrid

- Reuses OpenFHE internals, including Hybrid keyswitching settings
- Downsides:

- Implementation-specific: parameters tied to keyswitching, can be a problem if a program uses both keyswitching and RGSW
- Internal product not really valid
- Security must be evaluated at ciphertext modulus QP

Algorithm 3.3 RGSW Encryption

Input: Decomposition parameter ℓ, base B
Input: RNS polynomial $[\mu]^{\mathsf{NTT}}_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$
Output: $\mathsf{RGSW}(\mu)$

```
Initialize empty array C with 2kℓ slots
for r ← [0, kℓ) in parallel do
    z ← HE.Encrypt(0, pk_{B_QP})
    mG ← P · BaseExtend(m, Q, P)
    C_r ← HE.Enc(0) + (mg, 0)
    C_{r+ℓ} ← HE.Enc(0) + (0, mg)
end for
return C
```

3.1.3 SIMD

- Assuming parameters that support SIMD batching are set
- Requires using `MakePackedPlaintext` instead of `MakeCoefPackedPlaintext` in OpenFHE

3.1.4 Implementation Details of sPAR

- Uses a procedural programming paradigm, meaning the library implements the required functions but does not provide a full state-maintaining implementation of the server nor client.
- The benchmarks and testing framework is what actually composes the protocol, using the functions from this library.
- This was done so that the functionality is not tightly coupled to the benchmarking without spending time implementing actual communication which would require additional work: serializing data, error handling, and coordinating multiple machines.
- Does not implement the bucket sorting step: negligible impact (estimated < 100 ms per client)

3.1.5 Server Write Step

- Most important algorithm; does the actual homomorphic sorting

- Some changes compared to [DP25]
  - Splits the write step and pre-processing step
  - HasWrite is inverted i.e. "notHasWrite"
  - Uses external products to handle deep circuits as [DP25] recommends.
  - Not implementable using only external products: must use internal products
  - As the sever does not care about the privacy of its users, and we can assume it is actively attempting to break privacy, to manage noise **the server does not actually encrypt the initial entries of L, I and $hasNotWritten$.** They are in the correct form but generated using $\epsilon = 0, a = 0$. This has a very minor impact on noise but worth noting, major impact on setup phase as it does not spend any time on encrypting or generating random values.

Algorithm 3.4 `Server::Write`

Input: Reference to state matrices I and L
Input: Encrypted RGSW value V_r
Input: RGSW indices A_0, A_1 and A_2 where A_j is $\log \eta$ encrypted bits $\{c_{j,0}, \ldots, c_{j,\eta-1}\}$ for $j \in [0, 3)$
Output: Updates the state matrices I and L
Output: Encrypted boolean indicating wether the operation was successful

1: $\mathsf{hasNotWritten} \leftarrow \textsc{EncryptRGSW}(1)$
2: **for** $d \leftarrow \{0, 1, 2\}$ **do**
3: **for** $k \leftarrow \{0, 1, 2\}$ **do**
4: **for** $i \leftarrow [0, \eta)$ **do**
5: $h \leftarrow z_{i,d} \boxtimes I_{i,k} \boxtimes \mathsf{hasNotWritten}$
6: $L_{i,k} \leftarrow L_{i,k} + V_r \boxdot h$
7: $I_{i,k} \leftarrow I_{i,k} - h$
8: $\mathsf{hasNotWritten} \leftarrow \mathsf{hasNotWritten} + h$
9: **end for**
10: **end for**
11: **end for**
12: **return** hasNotWritten

Remark 3.2. From a programming perspective, the choice of $K = 3$ and $D = 3$ is arbitrary. The library allows a developer to configure different values for K and D as compile-time constants, with no impact on the size of the compiled binary/program. They are implemented as C++ generics, also known as template specialization.

3.2 Experimental Setup

sPAR

The design To simulate communication between parties, a single "all-knowing" orchestrator with access to the secret key.

- An "all-knowing" orchestrator performs the setup and 1 full round
- In `benchmark` dir
  - Takes the number of users η as an input
  - Every "user" generates a private key share, and corresponding public key share
  - The public key shares are all synthesized to make one large public key
  - For keeping track, each user has a unique ID
  - Each user encodes their ID in an RGSW using the public key, and sends that message to the server
  - Each user also sends 3 random indices
  - After receiving all messages, the server performs the write operation on each message
- Setup phase (considered "free")
  - Generate public key shares and attributes them to η virtual users
  - Hybrid setup phase is more complicated; requires additional MPC to generate a joint public key in QP used to encrypt RGSW ciphertexts

Unit Tests + Benchmarks

- To empirically evaluate correctness and catch bugs early, the functionality of the program is unit tested
- The RGSW encryption + TFHE operations are tested for plaintext 0 and 1, and a larger message
- SIMD and regular variants are tested
- For sPAR protocol, the number of users is parameterized over η
- `test` checks for correctness at every stage, no failures means the protocol can handle η users
- Even though q_i are randomly generated each time, its empirically clear there is no difference between runs, no matter what q_i are (as long as $\|Q\|$ is at the same bit size)
- Difference between benchmarks and unit tests:
  - Benchmarks are optimized for speed, and do not perform any correctness checks (which could affect performance). Functions only need to run once and total time can be extrapolated: for the client encryption/decryption which can be run in parallel over/independently by each user, and the

server/final decryption, the one-shot runtime *is* the correct runtime. For the server write, we only care about the per-user runtime anyway.
    - Unit tests are not optimized for speed (in the compilation etc.) and does perform correctness checks multiple times, at different levels; that way, the library provides reasonable diagnostics. Not just that an operation failed, but also *why*/where it failed.
    - Unit tests and benchmarks match in parameter and other code, so a successful unit test empirically validates correctness while the benchmarks provides real performance numbers in a production setting.

Choice of Parameters

- There are two interesting choices of parameters:
    - Small: Security parameter $\lambda \approx 128$ bits (2^λ)
    - Large: Fastest parameteres that allow SIMD operations in BGV [SV11]
- To determine parameters, I used the lattice estimator [APS15]
- Try

Hybrid

- GHS must evaluate security under ciphertext modulus QP
    - Since $|P| \approx |Q|$, security is approx. halved
    - Compensate by doubling the polynomial degree N shows empirically (via estimator) to bring it back to approximately the same level
- Three sets of parameters: 2 for security parity and shared for SIMD capable

Noise Analysis

- For testing RGSW products, error is checked directly
- Simulation/benchmark/testing environment always has access to the secret key

$$x = (c_0, c_1) = (c_1 s + \Delta m + \epsilon, c_1) \tag{3.1}$$

$$\implies \epsilon = c_0 - c_1 s - \Delta m, \quad m = \textsc{Decrypt}(x, s) \tag{3.2}$$

Chapter 4

Results and Discussion

I begin by examining

4.1 Parameters

As a baseline I target a security parameter of $\lambda \approx 128$ according to the lattice estimator of [APS15]. The benchmarks are using the parameters from Table 4.1.

Table 4.1: Base parameters for the

Parameter	Value
decomposition (ℓ, B)	$(7, 2^9)$
number of RNS moduli k	3
first mod size $\log q_0$	60
ciphertext modulus order $\log Q$	2^{120}
standard deviation σ	3.19
plaintext modulus t	256
polynomial degree N	4096
security parameter λ	113.6

- For BV, the base $B = \lfloor \max q_i \rfloor$ is chosen automatically so to be the smallest B that covers the largest RNS prime with ℓ digits when the program starts, because the q_i are randomly chosen.
- I found the best results was setting the first RNS prime q_0 to a larger value on the order of 2^{60}. The remaining primes are on the order of 2^{30}.
- The decomposition parameter is $\ell = 7$, which was the smallest ℓ that could handle the noise with these parameters.

4.2

4.3 Noise Analysis

- How many users can be supported with these parameters?
- d_{num} can probably be greater than 1, but this is not supported by the code yet (GHS is hard-coded at the RGSW level)

External Product

- Take a fresh RGSW and apply it to an RLWE, save the result as the new RLWE and repeat
- Key takeaway: Hybrid performs better than BV at same security settings
- Maybe a pointless section

Figure 4.1: RNS-BV noise growth in chained external products

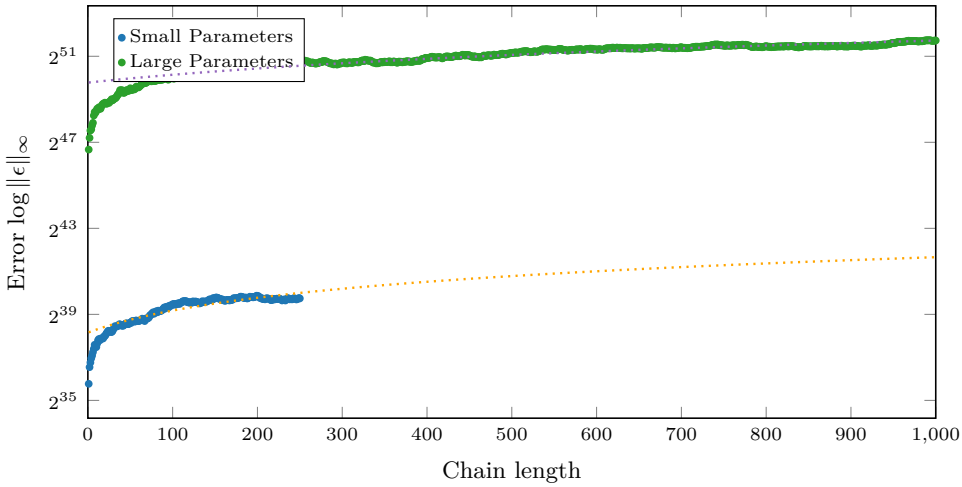
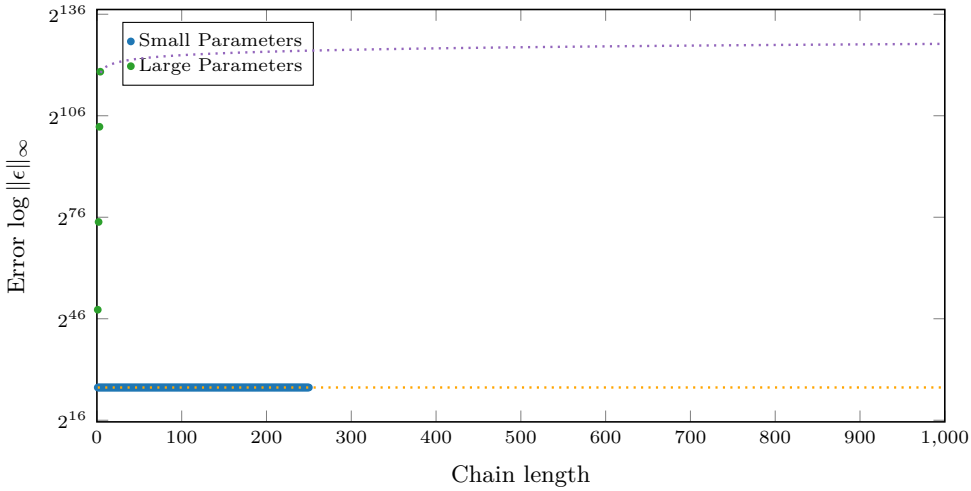


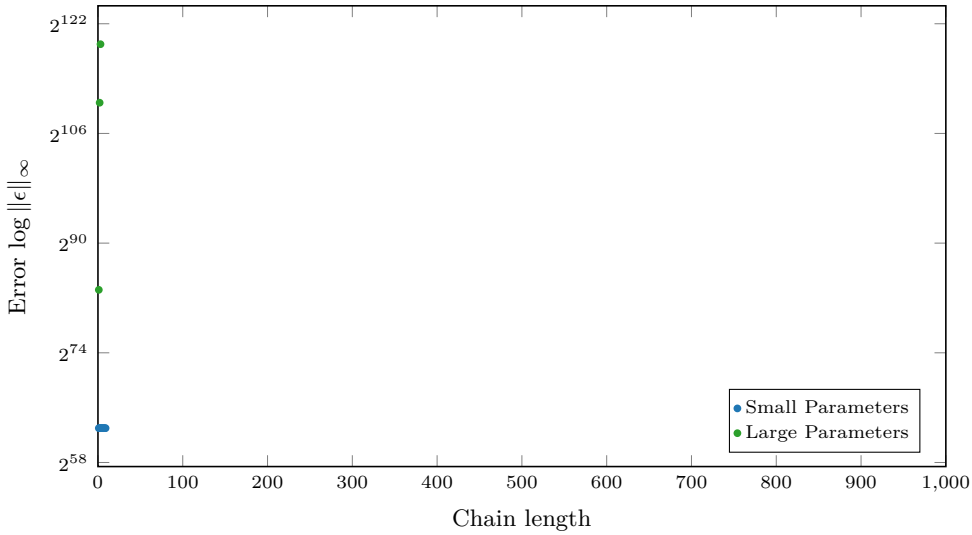
Figure 4.2: Hybrid noise growth in chained external products



Internal product

– Hybrid supports at most 1 single internal product before the noise blows up

Figure 4.3: RNS-BV noise growth in chained internal products



sPAR

- If the parameters support even the base case of $\eta = 2$ users, then the scheme can support a nearly arbitrary amount of users with the same parameters
- Noise is a big problem due to the internal product
- Getting enough noise headroom at $\lambda = 128$ requires very large parameters, making the protocol much slower than anticipated

4.4 Benchmarks

- Run on AMD Ryzen 5 5600X CPU 6-core @ 3.7 GHz with 32Gb RAM in WSL2 Ubuntu Linux
- Compare numbers with parity in the security parameter (according to the Lattice Estimator [APS15])
- Compare numbers at SIMD-capable level

4.4.1 RGSW Operations

Operation	Time
Encrypt RGSW	5.08 ms
External Product	1.66 ms
Internal Product	20.24 ms

(a) BV

Operation	Time
Encrypt RGSW	1.06 ms
External Product	0.70 ms
Internal Product	1.42 ms[a]

(b) GHS/Hybrid

[a] Not really valid since the noise is too large/result is incorrect...

Table 4.2: RGSW/TFHE Operations

- The internal product is $2k\ell$ external products, which performs $2k\ell$ multiplications $= \mathcal{O}(4k^2\ell^2)$ runtime
- Since GHS specifically has only 2 rows, the internal product $\mathcal{O}(2^2) = \mathcal{O}(1)$ has much better scaling — which would be impressive if it worked.

Multi-Threaded Performance

- Works now, but only some functions (EncryptRGSW, BV Decompose) are optimized to take advantage of multi-threading.
- $\leq k\ell$ threads saturate, so $k = 2, \ell = 3$ has optimal performance at 6 threads.
- Ideally more threads would used to perform multiple coefficients simultaneously, but this is not implemented.

Table of improvements +X% at 1 to 6 threads

sPAR

4.5 Discussions

The results show that

- Once noise is controlled, many users can be handled due to the additive scaling of the RGSW products
- GHS improvement over BV even adjusted to same security parameter, but would require a solution to the internal product

4.5.1 Theoretical Hybrid Improvement

- Assuming Algorithm 3.4 is implemented using only external products, or the internal product can be implemented in the hybrid scheme, we see that the hybrid approach is generally faster and has better noise properties

– These results are not technically valid as the noise overflows immediately after 1 or 2 internal products, but it gives an idea of what the system *could* do with more work

Table 4.3: One round of the sPAR protocol, using the **outdated** GHS gadget, single-threaded

η	**Total time**	**Encrypt**	**Write**	**Partial Dec.**	**Server Dec.**
2	147.05	7.36	59.42	6.36	0.78
32	37 431.27	121.61	966.33	77.63	38.25
64	144 160.94	231.51	1875.84	143.83	122.15
128	582 531.49	471.46	3776.87	296.88	486.43

Chapter 5
Conclusion and Future Work

Conclusion

- Achieved better time than expected with GHS
- Large noise
- Number of users

Future Work

- BGV SIMD capabilities [**<empty citation>**] not utilized in write loop; `Server::Write` can perform N rounds simultaneously, but does this translate into higher throughput?
- AVX512 not tested because time + no access to a computer that supports this instruction set; this would decrease the RNS primes to 48 bits, increasing performance and security, decreasing noise headroom
- RGSW/packed RLWE bandwidth optimization; originally planned, dropped due to time constraints, requires more complicated setup phase as $\mathsf{RGSW}s$ is required [CCR19; LY25]
- BFV implementation should be easy as the library sits on top of OpenFHE, it should be as easy as swapping out a few lines of code. Since BFV places its messages in the most significant bits, the decomposition gadget could be made inexact as with the powers of base, the least significant digits are dropped when ℓ is too small for B^ℓ to cover q
- The full sPAR protocols contains additional things such as equivocation protection, sorting buckets etc.

References

[APS15] M. R. Albrecht, R. Player, and S. Scott, *On the concrete hardness of learning with errors*, Cryptology ePrint Archive, Paper 2015/046, 2015. [Online]. Available: https://eprint.iacr.org/2015/046.

[BAB+22] A. A. Badawi, A. Alexandru, *et al.*, *OpenFHE: Open-source fully homomorphic encryption library*, Cryptology ePrint Archive, Paper 2022/915, 2022. [Online]. Available: https://eprint.iacr.org/2022/915.

[BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: https://eprint.iacr.org/2016/510.

[BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: https://eprint.iacr.org/2011/277.

[BV11] Z. Brakerski and V. Vaikuntanathan, *Efficient fully homomorphic encryption from (standard) LWE*, Cryptology ePrint Archive, Paper 2011/344, 2011. [Online]. Available: https://eprint.iacr.org/2011/344.

[CCR19] H. Chen, I. Chillotti, and L. Ren, "Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe", in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: https://doi.org/10.1145/3319535.3354226.

[CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: https://eprint.iacr.org/2018/421.

[DM14] L. Ducas and D. Micciancio, *FHEW: Bootstrapping homomorphic encryption in less than a second*, Cryptology ePrint Archive, Paper 2014/816, 2014. [Online]. Available: https://eprint.iacr.org/2014/816.

[DP25] D. Das and J. Park, *sPAR: (somewhat) practical anonymous router*, Cryptology ePrint Archive, Paper 2025/860, 2025. [Online]. Available: https://eprint.iacr.org/2025/860.

[Gen09] C. Gentry, "A fully homomorphic encryption scheme", Ph.D. dissertation, Stanford University, 2009. [Online]. Available: https://crypto.stanford.edu/craig.

[GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: https://eprint.iacr.org/2012/099.

[GSW13] C. Gentry, A. Sahai, and B. Waters, *Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based*, Cryptology ePrint Archive, Paper 2013/340, 2013. [Online]. Available: https://eprint.iacr.org/2013/340.

[HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: https://eprint.iacr.org/2018/117.

[KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: https://eprint.iacr.org/2021/204.

[LY25] K. H. Lee and J. W. Yoon, *Homomorphic field trace revisited : Breaking the cubic noise barrier*, Cryptology ePrint Archive, Paper 2025/1088, 2025. [Online]. Available: https://eprint.iacr.org/2025/1088.

[RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, "On data banks and privacy homomorphisms", in *Foundations of Secure Computation*, 1978, pp. 169–180.

[SEV+15] Y. Sun, A. Edmundson, *et al.*, "RAPTOR: Routing attacks on privacy in tor", in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun.

[SV11] N. P. Smart and F. Vercauteren, *Fully homomorphic SIMD operations*, Cryptology ePrint Archive, Paper 2011/133, 2011. [Online]. Available: https://eprint.iacr.org/2011/133.

[SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: https://eprint.iacr.org/2021/435.

Appendix A
Number Theory

A.1 Chinese Remaineder Theorem

Could be stated here just to be more complete

A.2 Number Theoretic Transform

- Turns a convolution into a nega/cyclic
- Same as FFT but over different ring